HACKFLASH PROBLEM STATEMENT 2

Healthcare Cost Intelligence

Technical Documentation

Abstract

This project investigates the factors associated with recorded healthcare billing using the supplied healthcare dataset. The analysis focuses on billing volume versus average billing per record and examines whether longer hospitalization corresponds to higher billing. After removing exact duplicate rows and excluding records with missing or negative billing from the primary cost analysis, the analytical dataset contains 54,858 records. Total recorded billing is 1,404,073,534.71, and average billing per record is 25,594.69. The Pearson and Spearman correlations between length of stay and billing are -0.0048 and -0.0047, respectively, indicating virtually no overall linear or monotonic association in this dataset. Results are descriptive of the supplied data and should not be generalized to real healthcare systems without validation.

1. Problem Statement

Healthcare Cost Intelligence: What is driving healthcare costs? Specifically, the analysis compares high total billing due to high patient volume versus high average billing per record. The required dimensions are billing amount, insurance provider, medical condition, hospital, admission type, length of stay, and medication. The problem statement also asks whether longer hospitalization corresponds to higher billing.

2. Objectives

• Audit and clean the dataset using documented, reproducible steps.
• Summarize total billing and average billing per record.
• Compare billing across insurance providers, medical conditions, admission types, medications, and hospitals. • Calculate length of stay from admission and discharge dates.
• Quantify the association between length of stay and billing.
• Communicate findings and limitations to support cautious, evidence-based decisions.

3. Dataset Overview

The source file contains 55,500 rows and 15 columns. Fields include patient demographics, medical condition, admission and discharge dates, doctor, hospital, insurance provider, billing amount, room number, admission type, medication, and test results. The source does not specify the billing currency or a clear reporting period; therefore, the report refers to billing amounts without assigning a currency or treating the total as an annual expenditure.

4. Methodology

The workflow consists of data loading, initial inspection, data-quality auditing, whitespace trimming, explicit day-first date parsing, duplicate removal, exclusion of records with missing or negative billing from the primary cost analysis, calculation of length of stay, exploratory summaries, visualizations, correlation analysis, and a one-way ANOVA comparing mean billing across medical conditions.

5. Data Quality Audit and Preprocessing

The supplied guidelines require checks for duplicate records, missing values, inconsistent formatting, invalid billing, invalid date sequences, implausible ages or stays, and inconsistent categories. The following table summarizes the checks performed and the resulting decisions.

Check / action	Observed result	Decision / interpretation
Original dataset	55,500 rows × 15 columns	Preserved as the source dataset.
Exact duplicates	534 rows removed	Kept one copy of each fully identical row.

Missing billing after deduplication 2		Excluded from primary cost analysis because billing is required.
Negative billing after deduplication 106		Excluded from primary cost analysis; source does not explain whether these are credits or errors.
Date parsing	0 invalid admission dates; 0 invalid discharge dates Parsed using %d/%m/%Y.	
Negative / zero stays		0 negative; 0 zero-day stays in reported audit Length of stay calculated as discharge date minus admission date.
Age validity	0 ages outside 0–120 in source	No values outside this broad range were found.
Casing audit		Name: 55,467; Doctor: 1,246; Hospital: 27,859 values differ from title case Audit flags are not automatically errors; names were not blindly rewritten.
Whitespace		Leading/trailing whitespace trimmed from text columns where present Changes logged by the notebook pipeline.

The working cost dataset retains non-negative billing records with non-missing billing. The final analytical dataset also requires valid admission and discharge dates and a non-negative length of stay. The cleaned master dataset is retained separately from the filtered analysis dataset so that exclusions are transparent.

Cleaning counts must be interpreted by stage: the initial source contained 108 negative billing values and 2 missing billing values, while after removing 534 duplicate rows, the cost-analysis exclusion counts were 106 negative and 2 missing billing records.

6. Descriptive Statistics

Metric	**Result**
Final analytical records	54,858
Total recorded billing	1,404,073,534.71
Average billing per record	25,594.69
Median billing per record	25,593.87
Average length of stay	15.50 days
Length of stay minimum–maximum	1–30 days
Unique hospital names	39,814
Hospitals represented by one record	32,025
Hospitals represented by five or fewer records	39,270

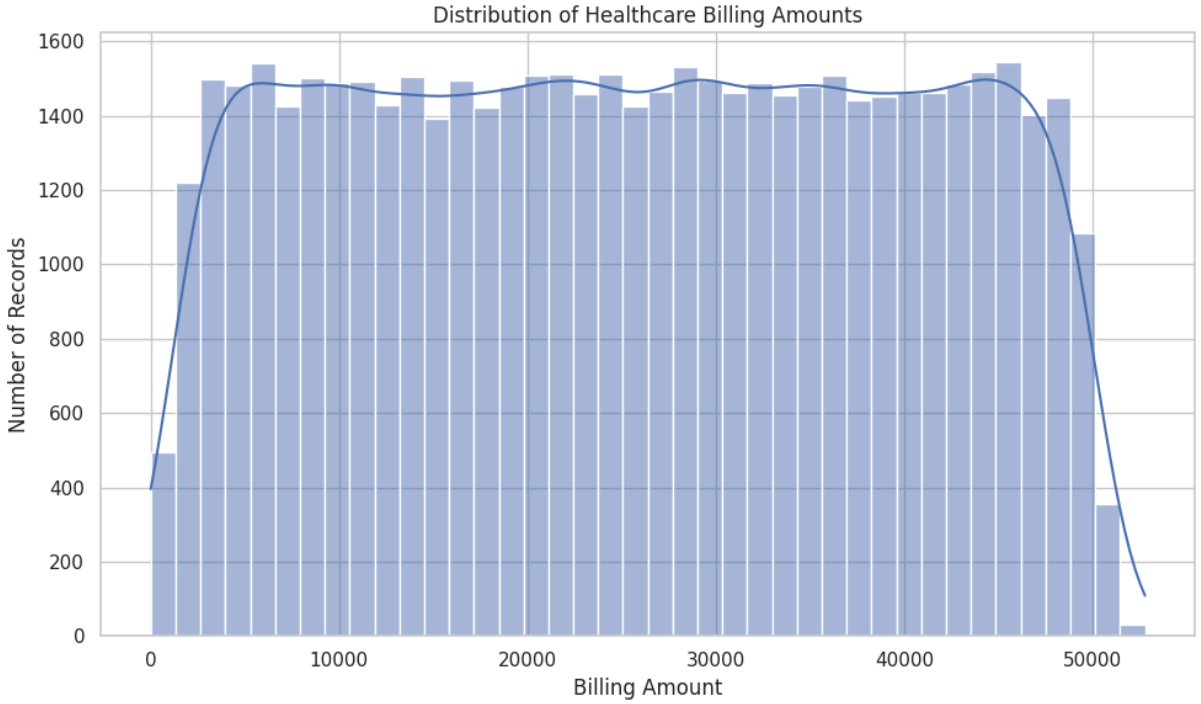


Figure 1. Distribution of billing amounts in the primary analytical dataset. The distribution spans a broad range; the coefficient of variation reported in the analysis is approximately 0.5539.Page 4

7. Exploratory Data Analysis and Visual Findings

7.1 Billing by Medical Condition

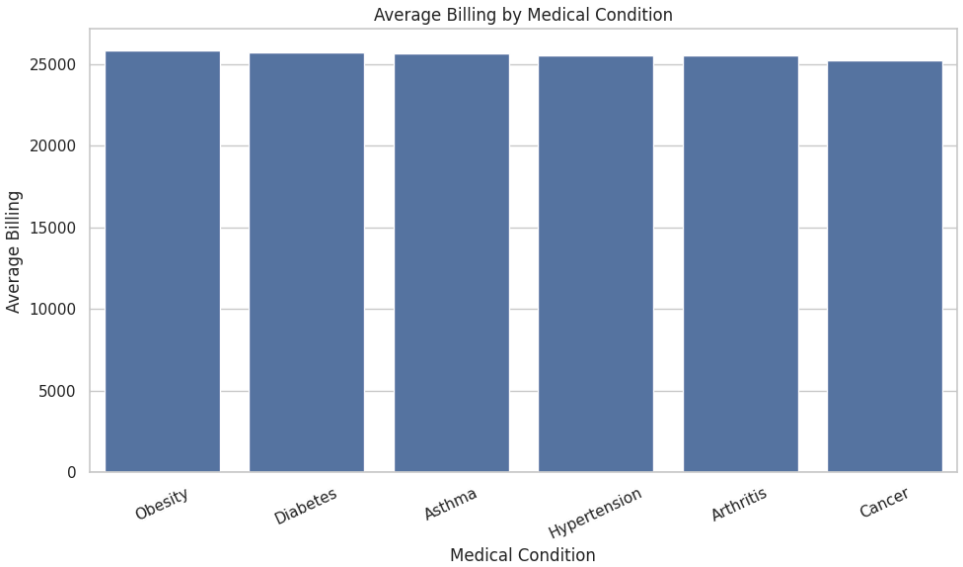


Figure 2. Average billing by medical condition. Obesity has the highest observed average (25,859.22), while Cancer has the lowest (25,205.92). The range is 653.29, equivalent to 2.55% of the overall mean billing.

7.2 Billing by Insurance Provider

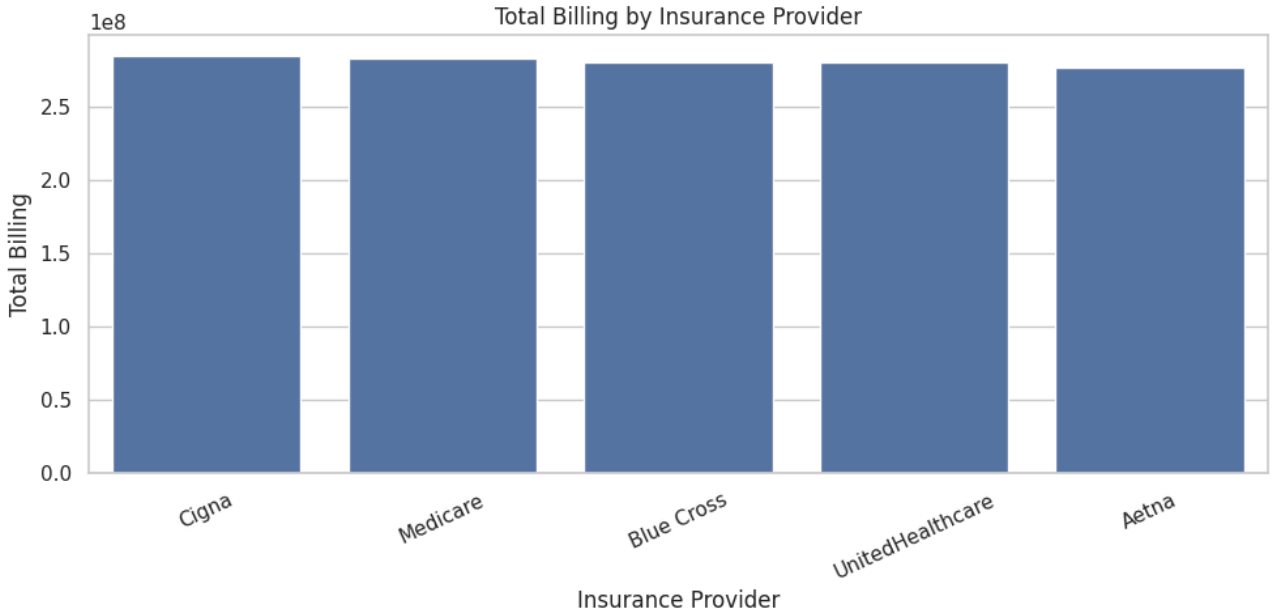


Figure 3. Total billing by insurance provider. Total billing is relatively close across the five providers; total billing alone reflects both patient-record volume and average billing.

7.3 Length of Stay and Billing

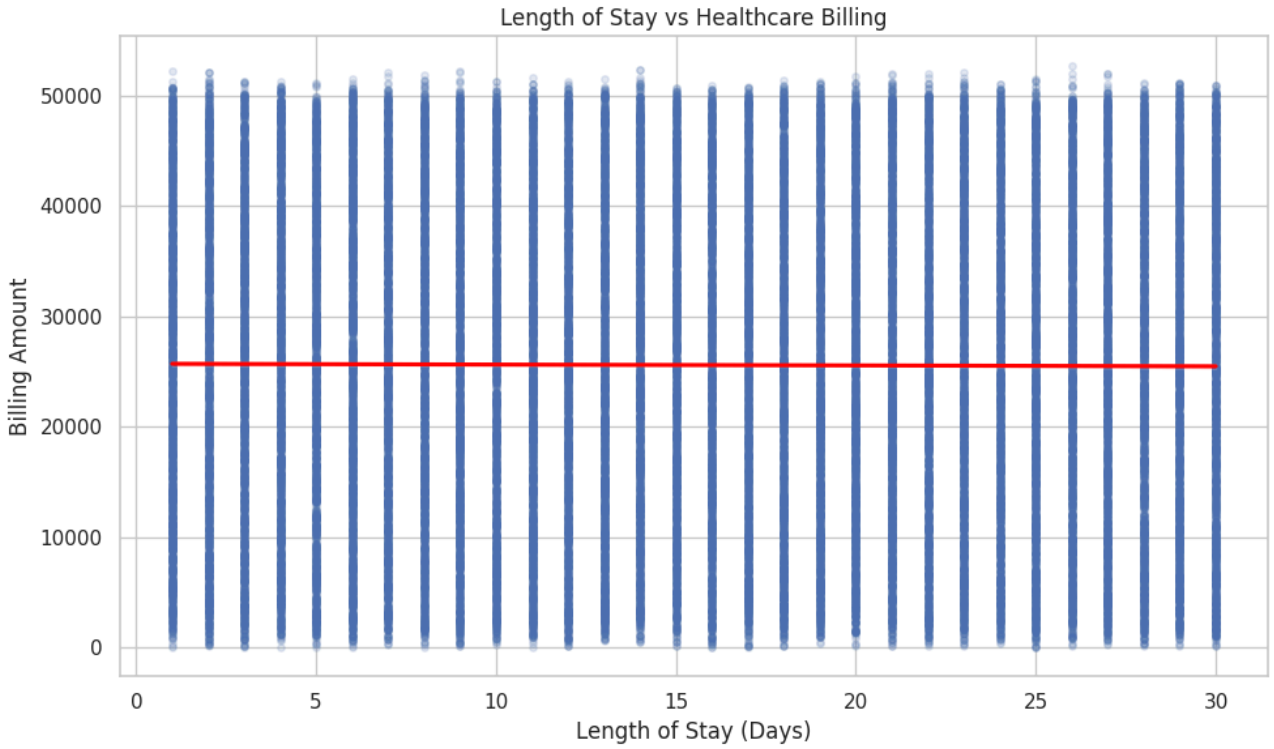


Figure 4. Individual billing amounts plotted against length of stay. Pearson correlation is -0.0048; Spearman correlation is -0.0047. Both are close to zero, indicating virtually no overall linear or monotonic association in this dataset.

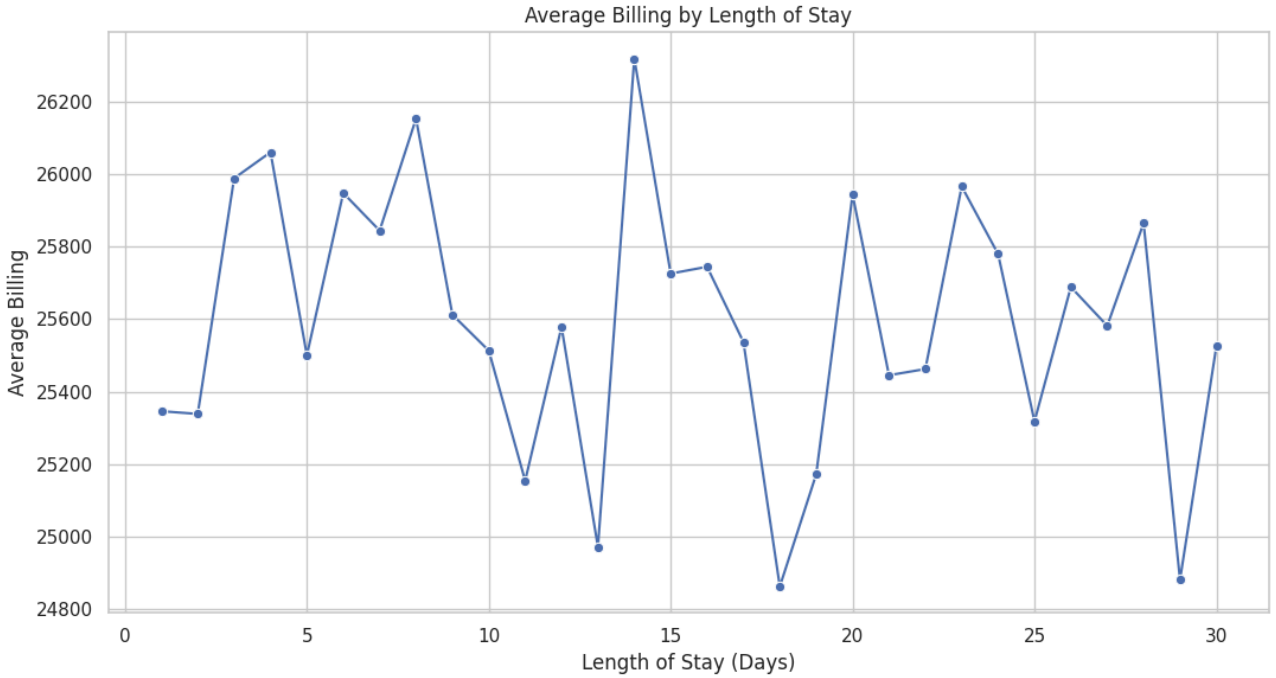
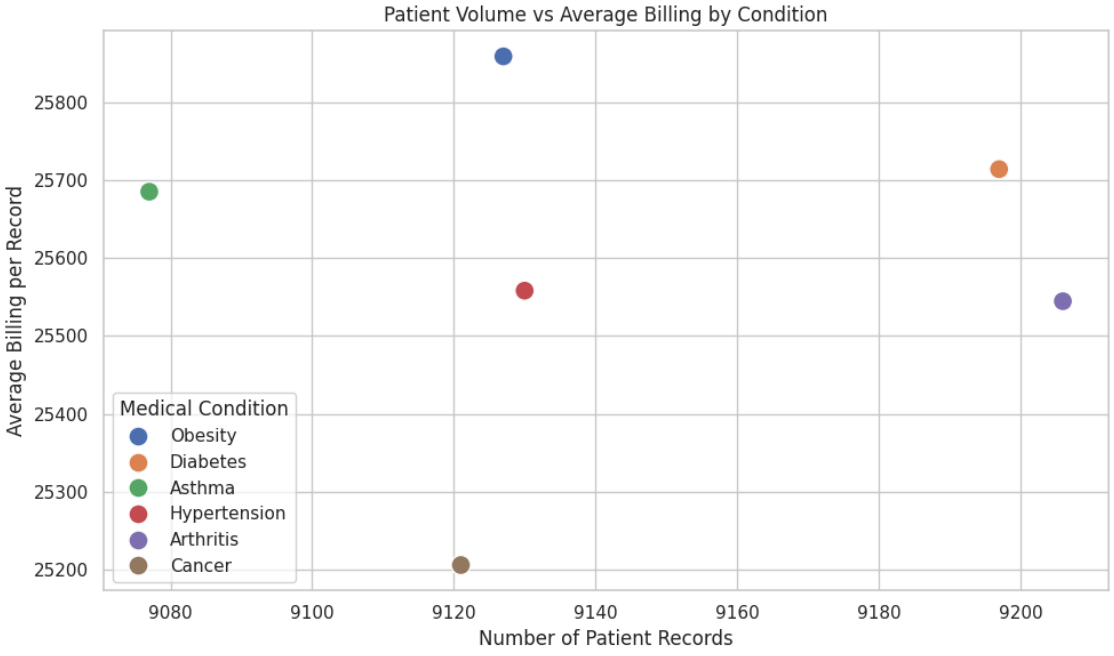


Figure 5. Average billing by length of stay. Means fluctuate across stay durations but do not show a consistent increasing trend.

7.4 Patient Volume versus Average Billing

For the six medical conditions, record counts range from approximately 9,077 to 9,206, while average billing differs by less than 3% of the overall mean. This suggests that within these condition categories, patient-record volume is similar and average billing is also relatively stable. Total billing differences should therefore be interpreted in light of both counts and averages.



8. Statistical Analysis

8.1 Correlation: Length of Stay and Billing

Pearson correlation: -0.0048. Spearman correlation: -0.0047. These results show essentially no linear or monotonic relationship between length of stay and billing in the analyzed records. Correlation does not establish causation, and this result should not be generalized beyond the supplied dataset.

8.2 One-Way ANOVA: Medical Condition and Billing

A one-way ANOVA compared mean billing across the six medical-condition groups. The F-statistic was 2.2481, with p = 0.04683. At the conventional 5% significance threshold, this is just below 0.05 and provides evidence that at least one group mean differs. The largest difference between condition-level average billing was 653.29 (2.55% of the overall mean). The p-value is borderline and the observed practical difference is modest. ANOVA does not identify which pairs differ; a post-hoc test would be needed for pairwise conclusions.

9. Hospital-Level Limitation

The dataset contains 39,814 distinct hospital strings across 54,858 records. 32,025 hospital names occur only once, and 39,270 occur five times or fewer. This extreme fragmentation makes rankings by hospital average or total billing unreliable for assessing hospital performance. Differences in capitalization may contribute to name fragmentation, but the supplied analysis does not establish that differently formatted names refer to the same facility. Hospital names should not be merged without a reliable identifier or verified mapping.

10. Key Findings

1. The final analysis contains 54,858 records with total recorded billing of 1,404,073,534.71 and average billing of 25,594.69 per record.
2. Length of stay has virtually no observed correlation with billing (Pearson r = -0.0048; Spearman r = -0.0047). 3. Average billing varies modestly across the analyzed category groups. The largest observed category-average range is across medical conditions, at 2.55% of the overall mean.
4. The ANOVA result for medical conditions is borderline significant (p = 0.04683), but the practical difference between group averages is small.
5. Hospital-level rankings are limited by a very high number of unique hospital names and many low-count groups.

11. Recommendations

• Treat this analysis as a descriptive review of the supplied dataset, not as proof of real-world healthcare cost drivers. • Do not use the current hospital ranking to reward or penalize facilities. Obtain stable hospital IDs and verify duplicate/variant facility names first.
• For cost management, track both total billing and average billing per record; also show patient volume beside totals. • Investigate the 106 negative billing records excluded after deduplication to determine whether they represent legitimate credits, reversals, or data errors.
• If available, add procedure-level charges, severity, diagnosis detail, service dates, and resource utilization to better explain cost differences.
• Review the borderline ANOVA finding with a post-hoc comparison only if identifying condition pairs is needed for a specific decision.

12. Limitations

The source does not specify billing currency, the exact observation period, or whether every row represents a unique patient or encounter. The report therefore uses “record” when describing counts. Excluding negative billing values may omit valid adjustments; these records were excluded because their meaning could not be verified. Category comparisons are descriptive and potentially confounded by factors not present in the dataset. Hospital names are highly fragmented. The results should not be interpreted as causal or generalized to real healthcare systems without data validation.

13. Conclusion

Total healthcare billing is determined by both the number of patient records and the average billing amount per record. The analysis compared these metrics across insurance providers, medical conditions, admission types, medications, and hospitals.

The results show that average billing differences across insurance providers, medical conditions, admission types, and medications are relatively small. Patient counts across the main categories are also broadly similar, and total billing across insurance providers is relatively comparable. Therefore, the analysis does not indicate a strong difference in

expenditure driven solely by higher average billing across these categories.

However, hospital-level comparisons are limited by the unusually large number of distinct hospital names and the small number of records associated with many of them. Consequently, the available evidence is insufficient to conclude definitively whether high total expenditure is primarily driven by patient volume or average billing at the hospital level.

A more reliable assessment would require standardized hospital identifiers and comparisons of patient volume, average billing, and total billing across hospitals with sufficient records.

Codes

The following appendix contains the code cells from the corrected Colab notebook, in execution order. The dataset upload cell is intended for Google Colab. The corrected notebook should be run from a fresh runtime to regenerate outputs before submission.

Code cell 2

```
# Import libraries for data analysis and visualization
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configure the appearance of plots
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

# Display more columns when inspecting dataframes
pd.set_option("display.max_columns", None)

print("Libraries imported successfully.")
```

Code cell 3

```
# Upload the healthcare CSV file into Google Colab
from google.colab import files

uploaded = files.upload()
```

Code cell 4

```
# Load the uploaded dataset into a pandas DataFrame
file_name = "healthcare_dataset.csv"

df = pd.read_csv(file_name)

# Display the first five records
display(df.head())
```

Code cell 5

```
# Check the number of rows and columns
print("Dataset shape:", df.shape)

# Inspect column names
print("\nColumn names:")
print(df.columns.tolist())

# Inspect data types and non-null counts
print("\nDataset information:")
df.info()

# Generate summary statistics for numerical columns
print("\nNumerical summary:")
display(df.describe())
```

Code cell 6

```
# Count missing values and calculate their percentage per column
missing_audit = pd.DataFrame({
 "Missing Count": df.isnull().sum(),
 "Missing Percentage": (
 df.isnull().mean() * 100
 ).round(2)
})
display(
 missing_audit.sort_values(
 by="Missing Count",
 ascending=False
 )
 )
```

Code cell 7

```
# Count fully duplicated rows

duplicate_count = df.duplicated().sum()

print("Fully duplicated rows:", duplicate_count)

# Inspect the duplicate records, if any
display(df[df.duplicated(keep=False)].head(10))
```

Code cell 8

```
# Check basic validity of age and billing amount
print("Age summary:")
display(df["Age"].describe())
print("\nAge values outside the range 0-120:") print(
 ((df["Age"] < 0) | (df["Age"] > 120)).sum() )
print("\nNegative billing amounts:")
print((df["Billing Amount"] < 0).sum())
print("\nZero billing amounts:")
print((df["Billing Amount"] == 0).sum())
```

Code cell 9

```
# Inspect unique values in categorical columns
categorical_columns = [
 "Gender",
 "Blood Type",
 "Medical Condition",
 "Insurance Provider",
 "Admission Type",
 "Medication",
 "Test Results"
]
for column in categorical_columns:
 print(f"\n{column}:")
 print(sorted(df[column].dropna().astype(str).unique()))
```

Code cell 10

```
# Preserve the original data before making any changes df_original = df.copy(deep=True)

# Store every cleaning action for transparent reporting cleaning_log = []

def log_cleaning(action, rows_affected, reason):  cleaning_log.append({
 "Action": action,
 "Rows Affected": int(rows_affected),
 "Reason": reason
 })
print("Original dataset preserved:", df_original.shape)
```

Code cell 11

```
# Audit inconsistent capitalization in text fields. # We inspect first, rather than blindly changing names.
for column in ["Name", "Doctor", "Hospital"]:
 values = df_original[column].dropna().astype(str)
 mixed_case_count = (
 values.ne(values.str.title())
 ).sum()
 print(
 f"{column}: {mixed_case_count} values "  "differ from title case"
 )
```

Code cell 12

```
# Create the working dataset
df_clean = df_original.copy()
# Remove leading/trailing whitespace from text fields text_columns = df_clean.select_dtypes(
 include=["object"]
 ).columns
for column in text_columns:
 before = df_clean[column].copy()
 df_clean[column] = df_clean[column].str.strip()

 changed = (
 before.notna() &
 df_clean[column].notna() &
 before.ne(df_clean[column])
 ).sum()
 if changed:
 log_cleaning(
 f"Trim whitespace in {column}",
 changed,
 "Remove accidental leading/trailing spaces."  )
# Parse dates explicitly using day/month/year format
for column in ["Date of Admission", "Discharge Date"]:
 df_clean[column] = pd.to_datetime(
 df_clean[column],
 format="%d/%m/%Y",
 errors="coerce"
 )
print("Date parsing complete.")
print(df_clean[[
 "Date of Admission",
 "Discharge Date"
]].dtypes)
print("\nInvalid or unparsed dates:")
print(
 df_clean[[
 "Date of Admission",
 "Discharge Date"
 ]].isna().sum()
 )
```

Code cell 13

```
# Remove fully duplicated records, keeping the first occurrence duplicates_before = df_clean.duplicated().sum()
df_clean = df_clean.drop_duplicates().copy()
log_cleaning(
 "Remove exact duplicate rows",
 duplicates_before,
 "Keep one copy of each fully identical record."
 )
print("Duplicate rows removed:", duplicates_before)
print("Shape after deduplication:", df_clean.shape)
```

Code cell 14

```
# Identify billing records that cannot be used in the
# primary analysis of positive healthcare charges

missing_billing = df_clean["Billing Amount"].isna()
negative_billing = df_clean["Billing Amount"] < 0

print("Missing billing:", missing_billing.sum())
print("Negative billing:", negative_billing.sum())

# Preserve all cleaned records in df_clean.
# Create a separate dataset for positive-billing analysis. df_cost = df_clean.loc[
 df_clean["Billing Amount"].notna() &
 (df_clean["Billing Amount"] >= 0)
].copy()

log_cleaning(
 "Exclude records with missing billing from cost analysis",  missing_billing.sum(),
 "Billing amount is required for cost calculations." )

log_cleaning(
 "Exclude records with negative billing from primary cost analysis",  negative_billing.sum(),
 "Investigate negative charges separately before interpreting costs." )

print("\nCleaned master dataset:", df_clean.shape)
print("Primary cost-analysis dataset:", df_cost.shape)
```

Code cell 15

```
# Calculate the difference between admission and discharge dates df_clean["Length of Stay"] = (
 df_clean["Discharge Date"] -
 df_clean["Date of Admission"]
).dt.days

# Count invalid dates and negative lengths of stay

invalid_date_rows = (
 df_clean["Date of Admission"].isna() |
 df_clean["Discharge Date"].isna()
)

negative_stay = df_clean["Length of Stay"] < 0
zero_stay = df_clean["Length of Stay"] == 0

print("Records with unparsed dates:", invalid_date_rows.sum()) print("Negative lengths of stay:", negative_stay.sum())
print("Zero-day stays:", zero_stay.sum())

# Display examples of negative stays for investigation
display(
 df_clean.loc[
 negative_stay,
 [
 "Date of Admission",
 "Discharge Date",
 "Length of Stay"
 ]
 ].head(10)
)
```

Code cell 16

```
# Create Length of Stay in the cost-analysis dataset.
# This cell makes df_analysis reproducible after a fresh runtime restart.

df_cost["Length of Stay"] = (
 df_cost["Discharge Date"] -
 df_cost["Date of Admission"]
).dt.days

# Keep records with valid dates and a non-negative stay duration. valid_stay = (
 df_cost["Date of Admission"].notna() &
 df_cost["Discharge Date"].notna() &
 (df_cost["Length of Stay"] >= 0)
)

excluded_stay_count = (~valid_stay).sum()
df_analysis = df_cost.loc[valid_stay].copy()

log_cleaning(
```

```
 "Exclude invalid dates or negative stays from stay analysis",  excluded_stay_count,
 "A valid non-negative length of stay is required."
 )
print("Cleaned master dataset:", df_clean.shape)
print("Cost-analysis dataset:", df_cost.shape)
print("Final analysis dataset:", df_analysis.shape)
display(df_analysis["Length of Stay"].describe())
```

Code cell 17

```
# Summary of healthcare billing and length of stay
summary = df_analysis[
 ["Billing Amount", "Length of Stay", "Age"]
].describe().round(2)
display(summary)
# Total and average billing
print("Total billing:", round(
 df_analysis["Billing Amount"].sum(), 2
 ))
print("Average billing per record:", round(
 df_analysis["Billing Amount"].mean(), 2
 ))
print("Median billing per record:", round(
 df_analysis["Billing Amount"].median(), 2
 ))
print("Average length of stay:", round(
 df_analysis["Length of Stay"].mean(), 2
 ), "days")
```

Code cell 18

```
# Plot the distribution of billing amounts
plt.figure(figsize=(10, 6))
sns.histplot(
 data=df_analysis,
 x="Billing Amount",
 bins=40,
 kde=True)
plt.title("Distribution of Healthcare Billing Amounts") plt.xlabel("Billing Amount")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
```

Code cell 19

```
# Aggregate hospital-level billing metrics
hospital_costs = (
 df_analysis.groupby("Hospital")
 .agg(
 Patient_Count=("Billing Amount", "size"),  Total_Billing=("Billing Amount", "sum"),
 Average_Billing=("Billing Amount", "mean"),  Median_Billing=("Billing Amount", "median")  )
 .reset_index()
 )
# Show hospitals with the highest total billing
top_total_billing = hospital_costs.nlargest(
 10, "Total_Billing"
 )
display(top_total_billing.round(2))
```

Code cell 20

```
# Compare the 10 hospitals with the highest total billing
plot_data = top_total_billing.sort_values(
 "Total_Billing", ascending=True
```

```
)
plt.figure(figsize=(12, 7))
sns.barplot(
 data=plot_data,
 x="Total_Billing",
 y="Hospital"
)
plt.title("Top 10 Hospitals by Total Billing")
plt.xlabel("Total Billing")
plt.ylabel("Hospital")
plt.tight_layout()
plt.show()
```

Code cell 21

```
# Identify hospitals with the highest average billing per record
top_average_billing = hospital_costs.nlargest(
 10, "Average_Billing"
)
display(top_average_billing.round(2))
# Plot the results
plot_data = top_average_billing.sort_values(
 "Average_Billing", ascending=True
)
plt.figure(figsize=(12, 7))
sns.barplot(
 data=plot_data,
 x="Average_Billing",
 y="Hospital"
)
plt.title("Top 10 Hospitals by Average Billing")
plt.xlabel("Average Billing per Record")
plt.ylabel("Hospital")
plt.tight_layout()
plt.show()
```

Code cell 22

```
# Summarize costs by insurance provider
insurance_costs = (
 df_analysis.groupby("Insurance Provider")
 .agg(
 Patient_Count=("Billing Amount", "size"),  Total_Billing=("Billing Amount", "sum"),
 Average_Billing=("Billing Amount", "mean"),
 Median_Billing=("Billing Amount", "median")  )
 .reset_index()
 .sort_values("Total_Billing", ascending=False) )
display(insurance_costs.round(2))
```

Code cell 23

```
# Total billing by insurance provider
plt.figure(figsize=(10, 5))
sns.barplot(
 data=insurance_costs,
 x="Insurance Provider",
 y="Total_Billing"
)
plt.title("Total Billing by Insurance Provider") plt.xlabel("Insurance Provider")
plt.ylabel("Total Billing")
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()
```

```
# Average billing per record by insurance provider plt.figure(figsize=(10, 5))
sns.barplot(
 data=insurance_costs,
 x="Insurance Provider",
 y="Average_Billing"
 )
plt.title("Average Billing per Record by Insurance Provider") plt.xlabel("Insurance Provider")
plt.ylabel("Average Billing")
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()
```

Code cell 24

```
# Calculate the correlation between length of stay and billing
pearson_corr = df_analysis[
 ["Length of Stay", "Billing Amount"]
 ].corr(method="pearson").iloc[0, 1]
spearman_corr = df_analysis[
 ["Length of Stay", "Billing Amount"]
 ].corr(method="spearman").iloc[0, 1]
print("Pearson correlation:", round(pearson_corr, 4)) print("Spearman correlation:", round(spearman_corr, 4))
```

Code cell 25

```
# Scatter plot with a fitted trend line
plt.figure(figsize=(10, 6))
sns.regplot(
 data=df_analysis,
 x="Length of Stay",
 y="Billing Amount",
 scatter_kws={"alpha": 0.15, "s": 15},
 line_kws={"color": "red"}
 )
plt.title("Length of Stay vs Healthcare Billing") plt.xlabel("Length of Stay (Days)")
plt.ylabel("Billing Amount")
plt.tight_layout()
plt.show()
```

Code cell 26

```
# Determine how many distinct hospitals exist
unique_hospitals = df_analysis["Hospital"].nunique()
print("Unique hospital names:", unique_hospitals) print("Total records:", len(df_analysis))
print(
 "Records per hospital summary:"
 )
display(
 df_analysis.groupby("Hospital")
 .size()
 .describe()
 )
# Check how many hospitals have very few records hospital_counts = df_analysis["Hospital"].value_counts()
print(
 "\nHospitals with 1 record:",
 (hospital_counts == 1).sum()
 )
print(
 "Hospitals with 5 or fewer records:",
 (hospital_counts <= 5).sum()
 )
print(
 "Hospitals with 20 or more records:",
```

```
 (hospital_counts >= 20).sum()
 )
```

Code cell 27

```
 # Compare billing across medical conditions
 condition_costs = (
  df_analysis.groupby("Medical Condition")  .agg(
  Patient_Count=("Billing Amount", "size"),  Total_Billing=("Billing Amount", "sum"),  Average_Billing=("Billing Amount",
 "mean"),  Median_Billing=("Billing Amount", "median")  )
  .reset_index()
  .sort_values("Average_Billing", ascending=False) )
 display(condition_costs.round(2))
 # Compare average billing across conditions
 plt.figure(figsize=(10, 6))
 sns.barplot(
  data=condition_costs,
  x="Medical Condition",
  y="Average_Billing"
 )
 plt.title("Average Billing by Medical Condition") plt.xlabel("Medical Condition")
 plt.ylabel("Average Billing")
 plt.xticks(rotation=25)
 plt.tight_layout()
 plt.show()
```

Code cell 28

```
 # Examine billing across admission types
 admission_costs = (
  df_analysis.groupby("Admission Type")
  .agg(
  Patient_Count=("Billing Amount", "size"),  Total_Billing=("Billing Amount", "sum"),  Average_Billing=("Billing Amount",
 "mean"),  Median_Billing=("Billing Amount", "median")  )
  .reset_index()
  .sort_values("Average_Billing", ascending=False) )
 display(admission_costs.round(2))
 # Visualize average billing by admission type plt.figure(figsize=(9, 5))
 sns.barplot(
  data=admission_costs,
  x="Admission Type",
  y="Average_Billing"
 )
 plt.title("Average Billing by Admission Type") plt.xlabel("Admission Type")
 plt.ylabel("Average Billing")
 plt.tight_layout()
 plt.show()
```

Code cell 29

```
 # Examine billing across medication categories
 medication_costs = (
  df_analysis.groupby("Medication")
  .agg(
  Patient_Count=("Billing Amount", "size"),  Total_Billing=("Billing Amount", "sum"),  Average_Billing=("Billing Amount",
 "mean"),  Median_Billing=("Billing Amount", "median")  )
  .reset_index()
  .sort_values("Average_Billing", ascending=False) )
 display(medication_costs.round(2))
 plt.figure(figsize=(10, 5))
 sns.barplot(
  data=medication_costs,
  x="Medication",
```

```
 y="Average_Billing"
 )
plt.title("Average Billing by Medication")
plt.xlabel("Medication")
plt.ylabel("Average Billing")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

Code cell 30

```
# Total billing can be decomposed into:
# number of records multiplied by average billing per record overall_volume = len(df_analysis)
overall_average_bill = df_analysis[
 "Billing Amount"
].mean()
reconstructed_total = (
 overall_volume * overall_average_bill
 )
print("Number of records:", overall_volume)
print("Average billing:", round(overall_average_bill, 2)) print("Reconstructed total billing:", round(
 reconstructed_total, 2
 ))
print("Actual total billing:", round(
 df_analysis["Billing Amount"].sum(), 2
 ))
```

Code cell 31

```
# Compare the spread of average billing across categories
category_summaries = {
 "Medical Condition": condition_costs,
 "Admission Type": admission_costs,
 "Medication": medication_costs,
 "Insurance Provider": insurance_costs
}
range_results = []
for category, table in category_summaries.items():  min_avg = table["Average_Billing"].min()
 max_avg = table["Average_Billing"].max()
 range_results.append({
 "Category": category,
 "Lowest Average Bill": min_avg,
 "Highest Average Bill": max_avg,
 "Difference": max_avg - min_avg,
 "Difference (% of overall mean)": (
 (max_avg - min_avg) /
 df_analysis["Billing Amount"].mean()  ) * 100
 })
range_df = pd.DataFrame(range_results)
display(range_df.round(2))
```

Code cell 32

```
# Examine average and standard deviation of billing # for each length of stay
stay_billing = (
 df_analysis.groupby("Length of Stay")
 .agg(
 Patient_Count=("Billing Amount", "size"),  Average_Billing=("Billing Amount", "mean"),  Median_Billing=("Billing
Amount", "median"),  Billing_Std=("Billing Amount", "std")
 )
 .reset_index()
 )
display(stay_billing.round(2))
# Plot average billing for each length of stay
```

```
plt.figure(figsize=(11, 6))
sns.lineplot(
 data=stay_billing,
 x="Length of Stay",
 y="Average_Billing",
 marker="o"
 )
plt.title("Average Billing by Length of Stay")
plt.xlabel("Length of Stay (Days)")
plt.ylabel("Average Billing")
plt.tight_layout()
plt.show()
```

Code cell 33

```
# Examine the number of distinct billing values
print(
 "Unique billing values:",
 df_analysis["Billing Amount"].nunique()
 )
print(
 "Total billing records:",
 len(df_analysis)
 )
# Compare quantiles
print("\nBilling quantiles:")
display(
 df_analysis["Billing Amount"]
 .quantile([0, 0.01, 0.10, 0.25, 0.50, 0.75, 0.90, 0.99, 1])  .to_frame(name="Billing Amount")
 )
# Check whether billing values are concentrated
# in a narrow or broad range
print(
 "\nCoefficient of variation:",
 round(
 df_analysis["Billing Amount"].std() /
 df_analysis["Billing Amount"].mean(),
 4
 )
 )
```

Code cell 34

```
# Compare patient volume and average billing
# across the medical conditions
plt.figure(figsize=(10, 6))
sns.scatterplot(
 data=condition_costs,
 x="Patient_Count",
 y="Average_Billing",
 hue="Medical Condition",
 s=150
 )
plt.title("Patient Volume vs Average Billing by Condition") plt.xlabel("Number of Patient Records")
plt.ylabel("Average Billing per Record")
plt.tight_layout()
plt.show()
```

Code cell 35

```
from scipy.stats import f_oneway
# Split billing amounts into groups by medical condition billing_groups = [
 group["Billing Amount"].values
 for _, group in df_analysis.groupby("Medical Condition") ]
# Test whether all group means are equal
f_statistic, p_value = f_oneway(*billing_groups)
```

```
print("ANOVA F-statistic:", round(f_statistic, 4))

print("ANOVA p-value:", p_value)

if p_value < 0.05:
 print(
 "Statistical conclusion: At least one condition's "
 "mean billing differs from another."
 )
else:
 print(
 "Statistical conclusion: Insufficient evidence of "
 "a difference in mean billing."
 )
```

Code cell 36

```
# Summarize the major results for the technical documentation

overall_mean = df_analysis["Billing Amount"].mean()
overall_total = df_analysis["Billing Amount"].sum()

findings_summary = pd.DataFrame([
 {
 "Analysis": "Total recorded billing",
 "Result": f"{overall_total:,.2f}",
 "Interpretation": "Total billing in the cleaned analysis dataset"  },
 {
 "Analysis": "Average billing per record",
 "Result": f"{overall_mean:,.2f}",
 "Interpretation": "Mean recorded billing per patient record"  },
 {
 "Analysis": "Average length of stay",
 "Result": f"{df_analysis['Length of Stay'].mean():.2f} days",  "Interpretation": "Mean calendar-day difference between
admission and discharge"  },
 {
 "Analysis": "Length of stay vs billing",
 "Result": f"Pearson r = {pearson_corr:.4f}",
 "Interpretation": "Practically no linear association"
 },
 {
 "Analysis": "Unique hospital names",
 "Result": f"{df_analysis['Hospital'].nunique():,}",
 "Interpretation": "Hospital-level comparisons are limited by fragmented names"  }
])

display(findings_summary)
```

—